Size Check / GarmentIQ Codebase Guide

Team handover documentation — architecture, live video & calibration deep dive, and a change cookbook

Generated from docs/CODEBASE_GUIDE.md

Size Check / GarmentIQ — Codebase Guide

A complete technical reference to this repository, written for handover to a new team. It covers what the project does, how every module fits together, exactly how the live-video camera pipeline and its calibration work, and a cookbook for the changes you're most likely to need. It assumes Python and basic computer-vision familiarity, but not prior exposure to this codebase.

This guide is deliberately long — read it top to bottom once, then use it as a reference. For step-by-step **installation** and **production-line operating instructions**, see the two companion guides it points to throughout:

- `docs/GTX1050TI_SETUP.md` (GTX1050TI_SETUP.md) — Windows/GPU installation and troubleshooting
- `docs/LIVE_PRODUCTION_LINE.md` (LIVE_PRODUCTION_LINE.md) — operating the underwear inspection station

Table of contents

1. What this project is
2. Repository map
3. End-to-end architecture
4. The core `garmentiq` package
5. The application layer (this fork)
6. Deep dive: how the live video pipeline works
7. Deep dive: calibration
8. Deep dive: automated underwear/production-line inspection
9. Deep dive: GPU / device management
10. Setup, running, and operating the demos
11. Testing
12. Cookbook: how to make common changes
13. Known limitations and non-goals
14. Appendix: key data structures

---

1. What this project is

The repository is a **fork of the open-source GarmentIQ package** (installed under `src/garmentiq/` (../src/garmentiq)), which measures garments from photos using computer vision: it classifies the garment type, segments it from the background, detects anatomical landmarks (waist corners, hems, etc.), and computes pixel distances between them.

On top of the upstream library, this fork adds an **application layer** — everything in the repository root and in `phone_web/` / `static_image_web/` — that turns the library into three runnable products:

Product	Entry point	What it does
Static photo upload demo	`static_image_server.py` (../static_image_server.py)	Upload one photo of a garment + ruler, calibrate by clicking, get measurements in inches.
Live camera / phone demo	`phone_server.py` (../phone_server.py)	Browser-based live camera feed, ruler calibration, and (new) **automatic paused-item underwear size inspection** for a production line.
OpenCV desktop window	`run_live.py` (../run_live.py)	A local webcam window with live-drawn measurements, no browser needed.

The fork's other major addition is **`src/garmentiq/underwear_sizes.py`**: a reference-matching size estimator (M/L/XL/2XL) for underwear, built on top of the library's existing `shorts` landmark model, plus frame-quality and geometry sanity checks used to decide when a live video frame is trustworthy enough to measure.

Everything needed to run the fork was also adapted for **Windows + an older, VRAM-limited GPU** (GTX 1050 Ti, 4 GB) — see `src/garmentiq/runtime.py` (../src/garmentiq/runtime.py) and Section 9.

> The original upstream GarmentIQ README (installation via PyPI, Colab notebooks, model training/fine-tuning workflows) is preserved below the fork-specific notes in the main `README.md` (../README.md) — it's still accurate for the `garmentiq` Python API itself.

---

2. Repository map

```
dimensions-check/
├── src/garmentiq/              The core Python package (pip-installable library, "the model")
│   ├── tailor.py               Orchestrator class: wires classify → segment → detect → refine → derive
│   ├── live.py                 LiveDetector: runs the tailor pipeline on live video/webcam frames
│   ├── runtime.py              Device selection + low-VRAM model offloading (GTX 1050 Ti support)
│   ├── underwear_sizes.py      M/L/XL/2XL reference matching + frame/landmark sanity checks (fork-only)
│   ├── garment_classes.py      Registry of the 9 supported garment types
│   ├── instruction/*.json      Per-garment landmark + measurement schemas (9 files)
│   ├── classification/         Garment-type classifier (CNN3 / CNN4 / tinyViT models)
│   ├── segmentation/           Background removal (BiRefNet / SAM models)
│   ├── landmark/                 detection/ (HRNet), refinement/, derivation/ submodules
│   └── utils/                  Small shared helpers (distance math, JSON export, validation)
│
├── phone_server.py              Flask app: live browser camera + underwear inspection API (this fork)
├── static_image_server.py       Flask app: static photo upload + calibration demo (this fork)
├── run_live.py                  Shared "build the tailor agent" logic + OpenCV desktop window launcher
├── run_live.bat / run_phone_pc.bat / run_phone_server.bat / run_static_demo.bat
│                                 Windows one-click launchers around the four entry points above
│
├── phone_web/                   Frontend for phone_server.py (live camera UI)
│   ├── index.html               Page layout/controls
│   ├── app.js                   Camera, calibration clicks, ROI, results rendering, HTTP calls
│   ├── inspection.mjs           InspectionGate: pure state machine deciding when to auto-capture
│   └── style.css
├── static_image_web/index.html  Single-file frontend for static_image_server.py
│
├── scripts/
│   ├── download_models.py       Downloads the 3 model weight files from Hugging Face, verifies SHA-256
│   └── check_runtime.py         GPU/driver/model smoke test used by setup and troubleshooting
│
├── models/                      Model weights (~722 MB, gitignored — see scripts/download_models.py)
├── demo_images/                 Sample image used by the smoke test
├── test/                        Python unittest files, Node test files, and upstream example notebooks
│
├── setup_gtx1050ti.bat / .ps1   One-shot Windows/CUDA 11.8 environment setup for this fork
├── requirements-gtx1050ti.txt   Pinned dependency set (CUDA 11.8 wheels) used by that setup
├── pyproject.toml               Package metadata for `garmentiq` (upstream PyPI package)
├── docs/
│   ├── GTX1050TI_SETUP.md       Windows/GPU install guide
│   ├── LIVE_PRODUCTION_LINE.md  Underwear inspection operating guide
│   └── CODEBASE_GUIDE.md        This file
└── README.md                    Fork notes + full upstream GarmentIQ documentation
```

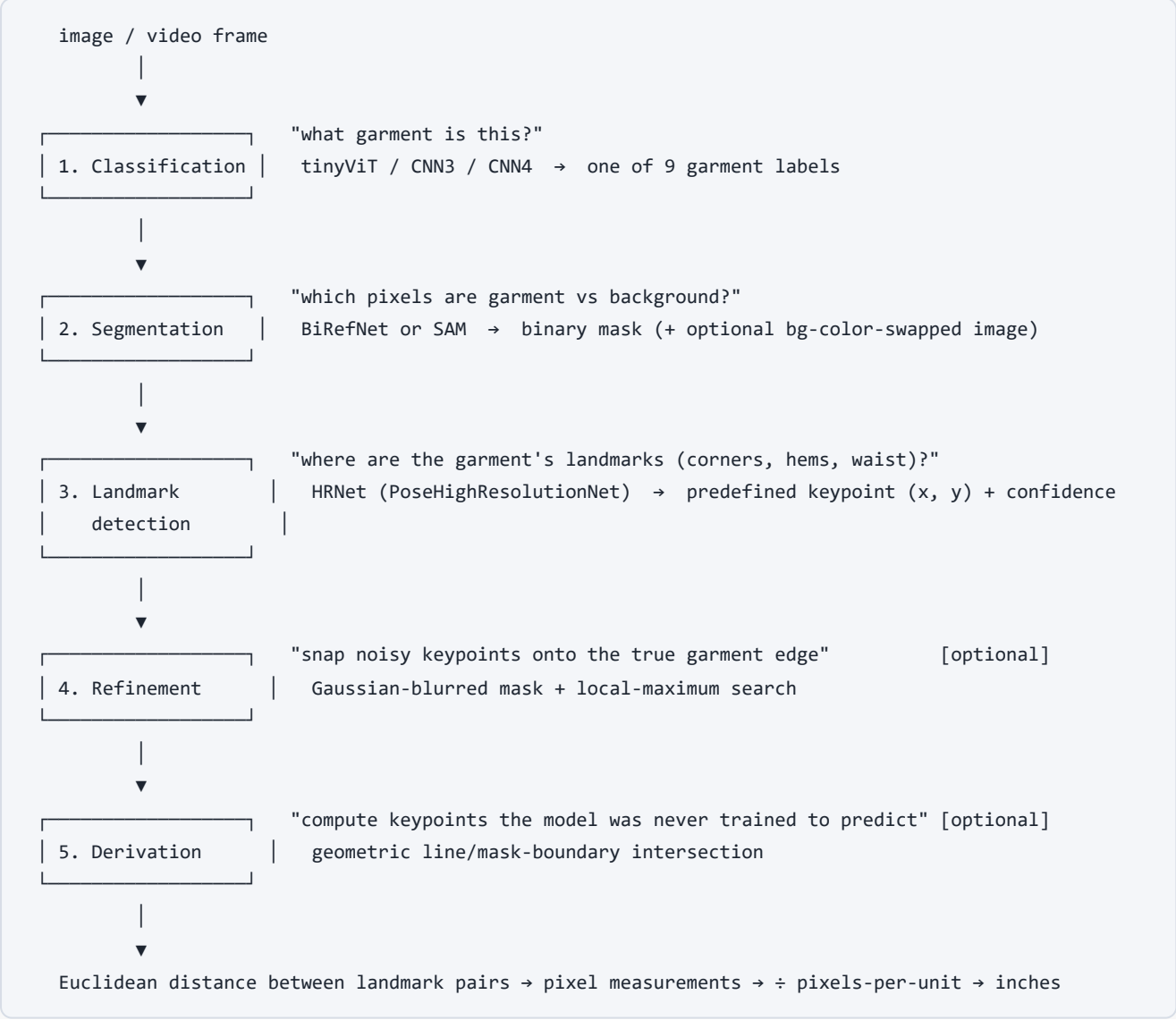
A useful mental split: `src/garmentiq/` **is the library** (what upstream publishes to PyPI as `garmentiq`); **everything else at the repo root is this fork's application**, built by importing that library. When in doubt about whether a change belongs in the library or the app layer, ask: "would this make sense to someone

using `pip install garmentiq` with no knowledge of underwear inspection or Flask servers?" If yes, it belongs under `src/garmentiq/`.

---

3. End-to-end architecture

Every product in this repo (static upload, live camera, desktop window) runs the **same five-stage pipeline** on each image/frame. The `tailor` class (batch mode, for folders of images) and `LiveDetector` class (single-frame mode, for video/camera) are two different drivers around the identical set of stages:



Which of stages 2, 4, 5 run is configurable per garment type via `do_derive` / `do_refine` flags and whether `background_color` is set — see `tailor.__init__` (../src/garmentiq/tailor.py) and Section 4.2.

The **instruction JSON files** in `src/garmentiq/instruction/` are what make each garment type's landmarks and measurements swappable without touching any model code — see Section 4.8.

---

4. The core `garmentiq` package

4.1 Package layout & import surface

`src/garmentiq/__init__.py` (../src/garmentiq/**init**.py) is intentionally tiny — it's the whole public API surface of the library:

```python
# garmentiq/__init__.py
from .tailor import tailor
from .live import LiveDetector, LiveResult
from . import utils
from . import classification
from . import segmentation
from . import landmark
```

Anything imported here is what `import garmentiq as giq` exposes as `giq.tailor`, `giq.LiveDetector`, `giq.classification.*`, etc. If you add a new top-level capability that other code (or notebooks) should call directly, export it here.

4.2 `tailor.py` — the pipeline orchestrator

`src/garmentiq/tailor.py` (../src/garmentiq/tailor.py) is the central class. One `tailor` instance owns all three loaded models (classification, segmentation, landmark detection) plus the garment-class registry and pipeline configuration, and exposes both:

- **Stage-by-stage methods** — `classify()`, `segment()`, `detect()`, `refine()`, `derive()` — each a thin, cleaned-up wrapper around the matching submodule function.
- **`measure()`** — a full batch run over every image in `input_dir`, saving JSON + annotated images to `output_dir`.

`__init__` loads all three models up front (CPU first, via each submodule's `load_model`), then hands them to `InferenceRuntime.prepare()` (Section 9) which decides whether to move them to GPU immediately or keep them staged in system RAM:

```python
self.runtime = runtime or InferenceRuntime(device, low_vram)
...
self.classification_model = classification.load_model(
    model_path=f"{self.model_dir}/{self.classification_model_path}",
    model_class=self.classification_model_class,
    model_args=filtered_model_args,
    device="cpu",
)
...
self.runtime.prepare([
    self.classification_model, self.segmentation_model, self.landmark_detection_model
])
```

`segment()` is worth reading closely because it's an "intelligent router": BiRefNet and SAM take different keyword arguments (`resize_dim` / `normalize_mean` / `normalize_std` vs `processor` / `input_points`), so it filters `segmentation_model_args` down to whatever the chosen backend actually needs, and only returns a third `bg_modified_img` element when `background_color` was configured:

```
extraction_kwargs = {
    k: v for k, v in self.segmentation_model_args.items()
    if k not in ["model_config", "background_color"]
}
original_img, mask = self.runtime.run(segmentation.extract, model=self.segmentation_model,
                                      image_path=image_input, **extraction_kwargs)
background_color = self.segmentation_model_args.get("background_color")
if background_color is None:
    return original_img, mask
else:
    bg_modified_img = segmentation.change_background_color(...)
    return original_img, mask, bg_modified_img
```

`measure()` (the batch entry point) is a 13-step procedure: create output folders → glob input images → classify all → segment all (if needed) → detect landmarks on all → refine (if enabled) → derive (if enabled) → save mask/bg images (if requested) → save annotated images (if requested) → save one cleaned JSON per image → save a `metadata.csv` index. It is what the upstream Colab notebooks and README quick-start examples call; the fork's live/phone servers do **not** call `measure()` — they call the individual stage methods directly per frame (see `static_image_server.run_original_pipeline` (../static_image_server.py) and `LiveDetector.process_frame` (../src/garmentiq/live.py)) because `measure()` is disk/folder-oriented and batch-shaped, not built for a single in-memory frame on a request/response cycle.

4.3 `runtime.py` — device & VRAM management

Covered in depth in Section 9; in one line: `select_device()` actually runs a tiny CUDA convolution (not just `torch.cuda.is_available()`) before trusting a GPU, and `InferenceRuntime` optionally keeps only one model resident on GPU at a time for 4 GB cards.

4.4 `live.py` — `LiveDetector`

`src/garmentiq/live.py` (../src/garmentiq/live.py) is the fork's addition that adapts the folder-batch `tailor` pipeline to **one frame at a time**, which is what every live/video/phone code path in this repo actually uses. Full walkthrough in Section 6.

4.5 `classification/` module

Identifies which of the 9 garment types (Section 4.8) is in the image.

File	Role
`model_definition.py` (../src/garmentiq/classification/model_definition.py)	`CNN3`, `CNN4` (plain conv nets) and `tinyViT` (a small Vision Transformer) — the three architectures GarmentIQ ships pretrained weights for.
`predict.py` (../src/garmentiq/classification/predict.py)	`predict(model, image_path, classes, resize_dim, normalize_mean, normalize_std)` — resize → tensor → normalize → softmax → argmax against the **sorted** class list. Returns `(label, probabilities)`.
`load_model.py` (../src/garmentiq/classification/load_model.py)	Instantiates `model_class(**model_args)`, loads a `.pt` checkpoint with `torch.load(..., weights_only=True)`, strips a `module.` prefix (for models saved from `DataParallel`), `.eval()`s it.
`load_data.py`, `train_test_split.py`	Splits a zipped labelled dataset (metadata CSV + images) into train/test folders — used only for training, not by the live apps.
`train_pytorch_nn.py`, `fine_tune_pytorch_nn.py`, `test_pytorch_nn.py`	K-fold training/fine-tuning/evaluation loops with early stopping. Fine-tuning supports freezing all layers except ones matching `unfreeze_patterns` (e.g. `["classifier", "fc"]`).
`utils.py`	`CachedDataset` — a `torch.utils.data.Dataset` that pre-loads all images into memory for fast repeated epochs.

The three deployed apps (`run_live.py`, both servers) all use **`tinyViT`** with the fine-tuned weights `tiny_vit_inditex_finetuned.pt` (see the model table in the main `README.md` for accuracy numbers). Swapping to `CNN3` / `CNN4` only requires changing `classification_model_class` and `classification_model_path` in `run_live.build_agent()` (../run_live.py) — no other code changes.

4.6 `segmentation/` module

Produces a binary mask separating the garment from the background — required input for refinement and derivation, and for producing the green-screen image some landmark detectors are more accurate on.

File	Role
`extract.py` (../src/garmentiq/segmentation/extract.py)	The one function both backends share. If a Hugging Face `processor` kwarg is supplied it takes the SAM path (`processor(image, ..., input_points=...)` → `model(**inputs)` → `post_process_masks`); otherwise it takes the plain-PyTorch path (resize/normalize/sigmoid → resize mask back to the **original image's pixel grid**). That resize-back step is what keeps ruler-pixel calibration valid even when BiRefNet internally processes a smaller/larger image (see Section 9).
`load_model.py` (../src/garmentiq/segmentation/load_model.py)	Like classification's, but also accepts `.safetensors` checkpoints and auto-converts a Hugging Face config object into a plain dict via `model_args.to_dict()`.
`change_background_color.py` (../src/garmentiq/segmentation/change_background_color.py)	`image_np[mask_np == 0] = background_color` — flat pixel replacement, used to give the landmark model a clean, high-contrast background (the fork's apps use bright green `[102, 255, 102]`).
`model_definition/birefnet/`	The BiRefNet architecture + config loader (`load_birefnet_config()`), a large (2200+ line) segmentation network. Used as the default backend in this fork.
`model_definition/sam/`	Meta's Segment Anything (ViT-b/l/h) wrapper + config/processor loaders. Needs a `input_points` prompt (a pixel coordinate believed to be inside the garment) instead of running fully automatically.

BiRefNet vs SAM: BiRefNet is what this fork's `run_live.py` / `phone_server.py` / `static_image_server.py` all use — it needs no manual prompt point and its `resize_dim` directly controls internal compute cost (256 → 1024, tunable via `--segmentation-size`; see Section 9). SAM is fully supported by the library (see the README quick-start) but requires supplying a pixel coordinate inside the garment (`input_points=[[[x, y]]]`), which the fork's apps don't currently provide — adopting SAM in the live apps would need a way to pick that point automatically or from the calibration UI.

4.7 `landmark/` module

The three landmark stages live under `src/garmentiq/landmark/`:

Detection — `detect.py` (../src/garmentiq/landmark/detect.py) + `detection/`:

```
(input_tensor, image_np, center, scale) = input_image_transform(image_path, scale_std, resize_dim, ...)
np_output_heatmap = model(input_tensor).detach().cpu().numpy()
preds_heatmap, maxvals = get_final_preds(
    np_output_heatmap[:, class_element["index_range"][0]:class_element["index_range"][1], :, :]
)
```

The detection model — `PoseHighResolutionNet` (../src/garmentiq/landmark/detection/model_definition.py) (HRNet, adapted from the HRNet-for-Fashion-Landmark-Estimation project) — is trained across **all** garment types at once and outputs one heatmap channel per possible keypoint across the whole DeepFashion2 keypoint space (294 channels total). `detect()` slices out only the channel range belonging to the classified garment (`garment_classes[class_name]["index_range"]`, e.g. `(158, 168)` for `shorts`), converts each channel's heatmap peak to an image-pixel coordinate via `get_final_preds` / `transform_preds`, then writes the coordinates and confidences into the instruction JSON's `landmarks` dict at only the indices marked `"predefined": true`.

Refinement — `refine.py` (../src/garmentiq/landmark/refine.py) + `refinement/refine_landmark_with_blur.py` (../src/garmentiq/landmark/refinement/refine_landmark_with_blur.py): a cheap post-processing pass that nudges each detected point onto the segmentation mask. It Gaussian-blurs the mask (`ksize`, `sigmaX`), then for every landmark with confidence > 0 searches a `window_size`-pixel box around the raw detection for the blurred mask's local maximum and snaps the point there. This corrects small systematic offsets from the detection model without needing a smarter (slower) model.

Derivation — `derive.py` (../src/garmentiq/landmark/derive.py) + `derivation/{derivation_dict.py, prepare_args.py, process.py, derive_keypoint_coord.py, line_intersect.py, mask_intersect.py}`: computes coordinates for landmarks the detector was **never trained to predict** — instruction entries marked `"predefined": false` with a `"derivation"` block instead of fixed `x` / `y`. `derive()` walks those entries, validates/normalizes their arguments against `derivation_dict` (`prepare_args.py`), then dispatches to the named function (`process.py`). Today there is exactly one derivation function, `derive_keypoint_coord`:

1. **Line 1** passes through `p1_id` in a direction either *parallel* or *perpendicular* to the line `p2_id → p3_id` (`direction` parameter).
2. **Line 2** is the line through `p4_id → p5_id`.
3. Compute the line–line intersection of Line 1 and Line 2 as a first estimate.
4. Extract the segmentation mask's boundary as a polygon (`_get_mask_boundary`, via `shapely`).
5. Find where Line 1 actually crosses that boundary (`_find_line_mask_intersections`).
6. Return whichever mask-boundary crossing is closest to the Line 1/Line 2 estimate — falling back to the pure line–line intersection if the mask has no boundary or no crossing.

This is how, for example, a garment's "full length" hem point — which HRNet was never trained to mark directly — is derived as "the point directly below the shoulder landmark, extended down until it hits the mask edge." `garment_classes["skirt"]["index_range"]` and friends define which raw model channels feed the *predefined* points; the JSON's `derivation` blocks define everything computed geometrically on top of them.

4.8 `garment_classes.py` — the instruction JSON schema

`src/garmentiq/garment_classes.py` (../src/garmentiq/garment_classes.py) is a plain dict — the central registry of the 9 supported garment types:

```
garment_classes = {
    "shorts": {
        "num_predefined_points": 10,
        "index_range": (158, 168),
        "instruction": str(instruction_dir / "shorts.json")
    },
    ...
}
```

- `index_range`: which slice of the landmark detection model's 294-channel output belongs to this garment. **This is fixed by the pretrained HRNet weights** — do not change it unless you retrain the detector.
- `num_predefined_points`: how many of those channels are real (vs. padding).
- `instruction`: path to a JSON file (one per garment) that is the actual configurable part — the set of named landmarks and named measurements. E.g. `instruction/shorts.json` (../src/garmentiq/instruction/shorts.json):

```
{
  "shorts": {
    "landmarks": {
      "1": {"predefined": true, "description": "waist_left", "x": 50, "y": 35},
      "2": {"predefined": true, "description": "waist_center", "x": 94, "y": 39},
      "7": {"predefined": true, "description": "crotch_apex", "x": 94, "y": 80}
    },
    "measurements": {
      "waist": {"landmarks": {"start": "1", "end": "3"}, "description": "/"},
      "front rise": {"landmarks": {"start": "2", "end": "7"}, "description": "/"}
    }
  }
}
```

Landmark keys are **1-indexed strings** matching `str(channel_index + 1)`. `x` / `y` in the checked-in file are just placeholder/reference coordinates — they get overwritten with real detected pixel coordinates every time `detect()` runs. Measurements are simply named pairs of landmark IDs; `utils.compute_measurement_distances` (../src/garmentiq/utils/compute_measurement_distances.py) fills in the Euclidean `distance` between each pair.

Any landmark can instead be `"predefined": false` with a `"derivation"` block (see Section 4.7) naming a function from `derivation_dict` and its geometric arguments, e.g.:

```
"20": {
  "predefined": false,
  "description": "hem_bottom",
  "derivation": {"function": "derive_keypoint_coord", "p1_id": 2, "p2_id": 8, "p3_id": 18,
                 "p4_id": 8, "p5_id": 18, "direction": "parallel"}
}
```

This JSON schema is the primary customization surface of the whole library — see the "Custom measurement instruction" (../README.md#custom-measurement-instruction) example in the main README, and the "Add or change a measurement" recipe in Section 12.

4.9 `utils/` module

Small, focused helpers shared across the pipeline (all exported from `utils/__init__.py` (../src/garmentiq/utils/**init**.py)):

Function	Purpose
`compute_measurement_distances(garment_dict)`	Euclidean distance between each measurement's `start` / `end` landmark pair; writes `distance` back into the dict.
`clean_detection_dict(class_name, image_name, detection_dict)`	Strips internal bookkeeping fields, computes distances, and reshapes to `{image_name: {landmarks, measurements, class}}` — the shape every server response and saved JSON uses.
`export_dict_to_json(data, filename)`	Plain `json.dump` with indentation, used by `tailor.measure()`.
`validate_garment_class_dict(class_dict)`	Sanity-checks a `garment_classes`-shaped dict before `detect()` trusts it.
`unzip`, `check_unzipped_dir`, `check_filenames_metadata`	Training-data preparation helpers (used by `classification.train_test_split`), not used by the live apps.

4.10 `underwear_sizes.py` — reference matching & sanity checks

Fork-only module; the heart of the automated M/L/XL/2XL sizing feature. Deep dive in Section 8.

---

5. The application layer (this fork)

Everything below sits **on top of** `src/garmentiq/` and is specific to this fork — it's what turns the library into the three runnable demos.

5.1 `run_live.py` — shared agent builder + desktop window

`run_live.py` (../run_live.py) has two jobs:

1. **`build_agent(device, low_vram, segmentation_size)`** — the single place that constructs a fully configured `giq.tailor` instance (model paths, BiRefNet/HRNet/tinyViT args, derivation dict, `do_derive=True`, `do_refine=True`). **Both Flask servers import and call this function** (`import run_live`), so there is exactly one place that defines "which models, which weights, which preprocessing settings" for the whole application. If you need to change a model, a preprocessing constant, or add a new garment default, change it here once.
2. A CLI (`main()`) that opens a **local OpenCV window** (`cv2.imshow`) showing the webcam with live-drawn landmarks/measurements — no browser, no calibration UI, units are pixels unless you pass `--pixels-per-unit` / `--unit` on the command line.

`add_runtime_arguments(parser)` is also shared — it's what gives every entry point (`run_live.py`, both servers) the same `--device`, `--low-vram`, `--segmentation-size` CLI flags.

5.2 `static_image_server.py` — photo upload demo

`static_image_server.py` (../static_image_server.py) is a small Flask app serving `static_image_web/index.html` (../static_image_web/index.html) (a single self-contained HTML file with inline CSS/JS — no build step). Flow:

1. Browser lets the user pick an image file and click two points on the ruler (0" and 12" marks) directly on a `<canvas>`; `pixelsPerInch` is computed client-side as `distance_between_clicks / 12`.
2. `POST /measure` with the image file, optional forced `class_name`, and `pixels_per_inch`.
3. `run_original_pipeline()` on the server runs the exact same five stages `tailor.measure()` would (classify → segment → detect → refine → derive), but **in memory, on one image, without touching disk** — it's effectively a hand-inlined, non-batch version of `tailor.measure()`'s inner loop.
4. `annotate()` draws landmark dots and measurement lines with OpenCV onto the original BGR frame, dividing every pixel distance by `pixels_per_inch` to get inches.
5. Response is JSON: `garment_class`, `measurements` (name → `{distance, unit, landmarks}`), and the annotated image as a base64 JPEG data URL.

This is the simplest of the three apps and the best starting point for understanding the request/response shape before reading `phone_server.py`.

5.3 `phone_server.py` — live camera & underwear inspection API

`phone_server.py` (../phone_server.py) is the more complex Flask app, serving `phone_web/` (../phone_web) and exposing two endpoints:

- **`GET /size-references`** — returns the underwear `REFERENCES` table and tolerance from `underwear_sizes.py`, so the frontend's reference table is always in sync with the server without

hardcoding numbers in JS.

- `POST /measure` — the single endpoint used by every camera frame, in both "measure one frame" mode and automatic underwear-inspection mode. Full request/response walkthrough in Section 6.

Two details worth knowing up front:

- `inference_lock = threading.Lock()`, acquired **non-blocking** (`inference_lock.acquire(blocking=False)`). If a frame is already being processed, a new request gets an immediate `429 The model is busy` instead of queueing — this deliberately prevents a backlog of stale frames from a moving conveyor belt piling up behind a slow CPU/GPU.
- It's launched two ways: `--http` (plain HTTP, only safe for `127.0.0.1` — used by `run_phone_pc.bat` so a camera attached to the same PC can get browser camera permission without HTTPS) or the default HTTPS with an ad-hoc self-signed cert (`ssl_context="adhoc"` — used by `run_phone_server.bat` so a **phone** on the LAN can get camera permission, which browsers only grant on `localhost` or a secure origin).

5.4 `phone_web/` — the live camera frontend

Four files, no build step, loaded directly by Flask's static file serving:

- `index.html` — layout: a `<video>` + `<canvas overlay>` pair, mode selector (underwear-automatic vs. single-frame-measure), the three setup buttons (Calibrate / Set area / Capture empty area), Start/Stop/Reinspect controls, and the results panel.
- `app.js` — all UI logic and state: camera enumeration/switching, freeze-frame click capture for both calibration and ROI selection, the polling loop that drives automatic inspection, and rendering results. Deep dive in Sections 6–7.
- `inspection.mjs` — `InspectionGate`, a **pure, dependency-free state machine** (exported so it can be unit-tested with plain Node — see `test/test_inspection.mjs` (../test/test_inspection.mjs)) that decides *when* a paused item is stable enough to submit for measurement. Deep dive in Section 8.
- `style.css` — presentation only.

5.5 `static_image_web/index.html`

A single self-contained HTML file (no separate JS/CSS files) mirroring `static_image_server.py`'s API. It also contains a **client-side-only** "expected range" table (`ranges` object) for non-underwear garment types/sizes — a rough sanity-check overlay unrelated to the server-side `REFERENCES` table, present only in this older upload page. Underwear entries in this page do read the same `M/L/XL/2XL` values as `underwear_sizes.py` (duplicated here, not fetched — unlike `phone_web/app.js`, which fetches them from `/size-references`). Keep this duplication in mind if you ever change underwear reference values (see the cookbook in Section 12).

5.6 Launcher scripts

Each `.bat` file is a two-line wrapper: `cd` to the repo root, then call the matching Python entry point with the project's virtual environment interpreter, forwarding any extra CLI args (`%*`) and pausing on error so a double-clicked window doesn't just vanish:

Script	Runs	Default bind
`run_static_demo.bat`	`static_image_server.py`	`127.0.0.1:8080` HTTP
`run_phone_pc.bat`	`phone_server.py --http --host 127.0.0.1 --port 8080`	`127.0.0.1:8080` HTTP
`run_phone_server.bat`	`phone_server.py`	`0.0.0.0:8443` HTTPS (ad-hoc cert, for LAN phones)
`run_live.bat`	`run_live.py`	N/A — opens an OpenCV window, not a server

All four forward `--device`, `--low-vram`, `--segmentation-size` (and any other flag) straight through, e.g. `.\run_static_demo.bat --device cuda --segmentation-size 256`.

---

6. Deep dive: how the live video pipeline works

This section traces one camera frame from the browser to a displayed measurement, end to end.

6.1 The browser never streams video to the server

The `<video>` element shows a **live local preview only** — nothing is uploaded continuously. What gets sent to the server is always exactly **one JPEG snapshot**, captured on demand:

```
// phone_web/app.js
function snapshot() {
  const shot = document.createElement('canvas'); shot.width = roi.w; shot.height = roi.h;
  shot.getContext('2d').drawImage(video, roi.x, roi.y, roi.w, roi.h, 0, 0, roi.w, roi.h);
  return shot;
}
```

This draws only the calibrated **inspection area (ROI)** — not the whole camera frame — into an off-screen canvas at its native pixel size (crop, never rescale; this is what keeps the ruler's pixels-per-inch value valid, see Section 7). That canvas is then encoded to a JPEG blob and POSTed as multipart form data:

```
async function inspect(token = null) {
  ...
  const shot = snapshot();
  const blob = await new Promise(resolve => shot.toBlob(resolve, 'image/jpeg', .92));
  const form = new FormData();
  form.append('frame', blob, 'inspection.jpg');
  form.append('pixels_per_inch', scale);
  form.append('mode', $('mode').value);           // 'underwear' or 'measure'
  form.append('class_name', isUnderwear() ? 'shorts' : $('className').value);
  const response = await fetch('/measure', {method: 'POST', body: form, signal: controller.signal});
  ...
}
```

There are exactly two ways `inspect()` gets called:

- **Manually** — the "Measure one frame" button, for single-shot measurement of any of the 9 garment types (non-underwear mode).
- **Automatically** — the `InspectionGate` state machine (Section 8) calling `inspect(token)` the instant it decides a placed item has been still long enough.

Only one request is ever in flight (`busy` flag); a five-minute `AbortController` timeout guards against a hung request without blocking the UI forever.

6.2 Server side: `POST /measure`

`phone_server.py` (../phone_server.py) decodes the JPEG with OpenCV, validates it, and — this is the key object — builds a fresh **`LiveDetector`** for that one request:

```
detector = LiveDetector(
    agent,                              # the shared, already-loaded tailor instance
    pixels_per_unit=pixels_per_inch,    # from the browser's ruler calibration
    unit_name="in",
    class_name=class_name,              # None = auto-classify; "shorts" when mode=underwear
    inspect_underwear=underwear,
)
if not inference_lock.acquire(blocking=False):
    return jsonify(error="The model is busy. Wait and inspect this item again."), 429
try:
    result = detector.process_frame(frame)
    ...
finally:
    inference_lock.release()
```

`LiveDetector` is cheap to construct (it just stores config — the expensive models live on the shared `agent`), so building one per request is fine; it's what lets one request be "underwear mode with the shorts model and inch calibration" and the next be "measure mode, auto-classify, pixel output" without any shared mutable state between requests.

6.3 `LiveDetector.process_frame` — one frame through the pipeline

`src/garmentiq/live.py` (../src/garmentiq/live.py) is the frame-oriented twin of `tailor.measure()`. Walking through it:

```python
def process_frame(self, frame_bgr: np.ndarray) -> LiveResult:
    started = time.perf_counter()
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)                    # OpenCV is BGR; the models exp
    label = self.class_name or self.agent.classify(rgb, verbose=False)[0]  # stage 1 (skippable if class_

    needs_mask = self.agent.do_derive or self.agent.do_refine
    use_bg = self.agent.segmentation_model_args.get("background_color") is not None
    mask = None
    detection_image = rgb
    if needs_mask or use_bg or self.inspect_underwear:
        segmented = self.agent.segment(rgb)                             # stage 2
        mask = segmented[1]
        if use_bg:
            detection_image = segmented[2]                              # green-screened image, better
    if self.inspect_underwear:
        check_inspection_mask(mask)                                     # fork-only: reject bad frames

    coords, confidences, detection = self.agent.detect(label, detection_image)  # stage 3
    if self.agent.do_refine:
        coords, detection = self.agent.refine(label, coords, confidences, detection, mask)  # stage 4
    if self.agent.do_derive:
        _, detection = self.agent.derive(label, detection, self.agent.derivation_dict, coords, mask)  # s

    cleaned = utils.clean_detection_dict(label, "live_frame", detection)
    _, cleaned = utils.compute_measurement_distances(cleaned)
    garment = cleaned["live_frame"]
    if self.inspect_underwear:
        check_inspection_landmarks(garment["landmarks"])                # fork-only: reject bad geome

    measurements = copy.deepcopy(garment["measurements"])
    if self.pixels_per_unit:
        for measurement in measurements.values():
            measurement["distance"] /= self.pixels_per_unit
            measurement["unit"] = self.unit_name
    ...
    annotated = self._draw(frame_bgr.copy(), garment["landmarks"], measurements)
    return LiveResult(frame=annotated, garment_class=label, measurements=measurements,
                      detection=detection, inference_seconds=time.perf_counter() - started)
```

Notice it only runs segmentation **at all** if something actually needs the mask — a plain "measure this vest, no refinement/derivation configured" request skips stage 2 entirely to save time. The `run_live.build_agent()` configuration used by every app in this repo sets `do_derive=True` and `do_refine=True`, so in practice all three apps always segment.

`check_inspection_mask` / `check_inspection_landmarks` (from `underwear_sizes.py`, Section 8) are called **only** when `inspect_underwear=True`, and they `raise InspectionRejected(...)` rather than returning a value — `phone_server.py`'s `/measure` handler catches that specific exception and turns it into a normal `200` response with `size_estimate={"status": "uncertain", ...}` rather than an HTTP error, so a bad automatic frame degrades gracefully instead of erroring the page:

```
try:
    result = detector.process_frame(frame)
    measurements = result.measurements
    decision = estimate_size(measurements) if underwear else None
    annotated = result.frame
    label = result.garment_class
except InspectionRejected as exc:
    measurements, annotated, label = {}, frame, "shorts"
    decision = uncertain(str(exc))
```

`LiveDetector` also has a `.run()` method used only by `run_live.py`'s desktop window — it owns an OpenCV `cv2.VideoCapture` loop, calls `process_frame()` every `process_every_n_frames` frames, and shows the annotated result with `cv2.imshow` until Q/Escape. This is a genuinely different code path from the two Flask servers (which never open a camera themselves — the *browser* owns the camera and uploads snapshots) — useful to know if you're debugging "why does the desktop window behave differently from the browser demo."

6.4 Response and rendering

The server re-encodes the annotated BGR frame back to JPEG and returns JSON:

```
return jsonify(
    garment_class=label,
    inference_seconds=round(time.perf_counter() - started, 2),
    measurements=measurements,
    size_estimate=decision,           # null outside underwear mode
    image="" + base64.b64encode(jpeg).decode("ascii"),
)
```

`app.js`'s `renderResult()` puts that image into the results `<img>`, fills the measurements table, and — critically — **only after re-checking `current()`**, a guard that compares the request's captured epoch/gate-token against the live ones. This is what stops a slow response for an item that has since been removed from ever being shown as "the current result" — see Section 8 for exactly how that token system works.

---

7. Deep dive: calibration

Every measurement in this repository is ultimately a **pixel distance divided by a pixels-per-inch scale factor**. There is no camera intrinsics/extrinsics calibration, no lens distortion correction, and no perspective correction — calibration here means exactly one number: *how many pixels correspond to a real 12 inches*, measured once by the operator clicking two points on a physical ruler laid in the same plane as the garment.

7.1 Ruler calibration — the click-to-scale math

In `phone_web/app.js` (../phone_web/app.js), clicking "1. Calibrate 12-inch ruler" freezes the current video frame onto a canvas (`freezeFor('ruler')`) so the operator can click precisely without the image moving under their cursor. The next two clicks are captured by:

```
overlay.addEventListener('pointerdown', event => {
  if (!selection) return;
  const rect = overlay.getBoundingClientRect();
  const point = {x: ..., y: ...};   // converts CSS pixel click → canvas pixel coordinate
  points.push(point);
  if (points.length < 2) return;
  const [a, b] = points;
  if (selection === 'ruler') {
    const distance = Math.hypot(b.x - a.x, b.y - a.y);
    if (distance < 60) { /* reject: too close together, likely a misclick */ }
    scale = distance / 12;   // pixels per inch
  }
  ...
});
```

That's the entire calibration: **Euclidean pixel distance between the two clicked points, divided by 12** (a 12-inch ruler). `scale` (pixels/inch) is sent on every subsequent `/measure` request as `pixels_per_inch`, and `LiveDetector.process_frame` divides every measured pixel distance by it (`measurement["distance"] /= self.pixels_per_unit`). The static-image and phone pages both work this identical way; only the click target differs (canvas coordinates of an uploaded image vs. of a frozen live-camera frame).

The 60-pixel minimum-distance guard exists purely to catch accidental double-clicks on nearly the same spot, which would otherwise produce a wildly wrong (huge) pixels-per-inch value and silently corrupt every subsequent measurement.

This is a one-dimensional scale, not a full camera calibration — it assumes the garment and ruler are coplanar, the camera looks straight down (no meaningful perspective/lens distortion across the frame), and the camera never moves after calibration. See `docs/LIVE_PRODUCTION_LINE.md` (LIVE_PRODUCTION_LINE.md#quality-checks-and-limitations) for the operational implications and why the team should verify scale near the center *and* edges of the frame with a physical tape measure before trusting the whole inspection area.

7.2 The inspection area (ROI)

"2. Set inspection area" reuses the exact same freeze-frame, two-click flow, but interprets the two points as **opposite corners of a rectangle** instead of a ruler span:

```
} else {
  const w = Math.abs(b.x - a.x), h = Math.abs(b.y - a.y);
  if (Math.min(w, h) < 64) { /* reject: area too small */ }
  roi = {x: Math.min(a.x, b.x), y: Math.min(a.y, b.y), w, h};
}
```

`roi` is what `snapshot()` (Section 6.1) crops out of the live video before every measurement, and what `sample()` reads from for the presence/motion probes (Section 8). **Cropping never rescales pixels** — it's a pure sub-rectangle copy — so a pixels-per-inch value calibrated against the full frame stays exactly correct inside the cropped ROI. If `roi` were ever resized/scaled instead of cropped, the calibration would silently become wrong; this invariant is why the code comment in `inspect()` explicitly calls it out: `// Crop only: no rescaling, so ruler pixels/inch stays valid.`

The default ROI (before the operator sets one) is the whole camera frame: `roi = {x: 0, y: 0, w: video.videoWidth, h: video.videoHeight}` (set in `resetSetup()`).

7.3 The empty-area reference

"3. Capture empty inspection area" takes one more RGB sample — this time from the small, downscaled `probe` canvas (96×72, see Section 8.1) rather than the full ROI — with the ruler, garment, and hands all removed:

```
$('background').onclick = () => {
  background = sample(); epoch++; clearResult('Ready');
  ...
};
```

This `background` sample is the baseline every later frame is compared against to decide "*is anything even in the inspection area right now?*" — it is what `InspectionGate.update()`'s `foreground` fraction is measured relative to (Section 8.1). It is intentionally captured **separately from and after** ROI selection, because the ROI itself changes what pixels are being compared; **any change to calibration or the ROI clears it** (`freezeFor()` sets `background = null`), forcing the operator to recapture it, since an empty-reference from the old area/framing would silently make presence detection wrong.

7.4 What invalidates calibration, and how the page reacts

Calibration state (`scale`, `roi`, `background`) lives only in page memory — nothing persists across a reload. `app.js` watches for several conditions that make the existing calibration untrustworthy and forces a reset back to "recalibrate" (`resetSetup(message)`), each pointing the operator at a specific message:

Trigger	Detected by	Message
Camera resolution changes	`video`'s `resize` event	"Camera resolution changed. Recalibrate..."
Video frozen / stopped advancing	`tick()` comparing `video.currentTime` across polls	"Camera feed froze. Check the camera..."
Camera track ends (unplugged)	`stream.getVideoTracks()[0].onended`	"Camera disconnected. Reload this page..."
Page hidden (tab switched away)	`visibilitychange`	Inspection paused, not just calibration

Physical events the page **cannot** detect — someone bumping the camera, changing zoom, moving the belt height, or changing lighting — are exactly why `docs/LIVE_PRODUCTION_LINE.md` (LIVE_PRODUCTION_LINE.md) tells operators to manually stop and redo calibration + empty-capture after any such change. This is a real operational gap, not an oversight to silently "fix" — flag it in training material for whoever runs the line.

---

8. Deep dive: automated underwear/production-line inspection

This is the newest feature in the fork (see the `LIVE_PRODUCTION_LINE.md` guide for the operator POV). Two halves cooperate: a **browser-side trigger** (when is an item still enough to measure?) and a **server-side reference matcher** (given real measurements, what size — if any — do they match?).

8.1 Browser side: presence and motion detection

Every 250 ms while inspection is running, `tick()` samples the inspection area down to a tiny 96×72 probe canvas and compares it against two baselines:

```
// phone_web/inspection.mjs
export function changedFraction(a, b, threshold = 22) {
  if (!a || !b || a.length !== b.length || !a.length) return 1;
  let changed = 0;
  for (let i = 0; i < a.length; i += 4) {
    if ((Math.abs(a[i] - b[i]) + Math.abs(a[i + 1] - b[i + 1]) + Math.abs(a[i + 2] - b[i + 2])) / 3 > th
  }
  return changed / (a.length / 4);
}
```

This is a per-pixel average-RGB-difference test (threshold 22/255) counted as a **fraction of changed pixels** across the whole probe — deliberately crude/cheap so it can run 4×/second in plain JavaScript with no model inference. Two such fractions drive the state machine:

- `foreground = changedFraction(sample, background)` — how different is the area from the captured *empty* reference? ≥ 5% means "something is here."
- `motion = changedFraction(sample, previous)` — how different is the area from the *previous* probe (250 ms ago)? > 1.5% means "it's still moving." An **anchor** sample (captured the instant something first looked stable) is also compared against every subsequent probe, so slow cumulative drift across many small steps — which frame-to-frame comparison alone would miss — still counts as motion.

8.2 `InspectionGate` — the state machine

`phone_web/inspection.mjs` (../phone_web/inspection.mjs)'s `InspectionGate.update()` turns those two fractions into one of six states, called once per 250 ms tick:

```
      empty (<5% foreground)
        |  garment placed
        ▼
armed? ──no──► clear (already captured this item; wait for it to be removed)
  |yes
  ▼
moving (>1.5% change vs previous OR anchor)
  | stops moving
  ▼
settling (stable, but < 1250 ms so far)
  | 1250 ms of continuous stillness
  ▼
capture  ──► inspect(token) fires, armed=false, valid=true
  |
  ▼
holding (still present, already captured — waiting for removal)
  | area goes empty for ≥ 750 ms
  ▼
armed again (empty state, ready for the next item)
```

Two invariants matter for correctness:

- **Only one `capture` fires per placed item.** The instant `capture` fires, `armed` flips to `false`; every subsequent tick while the same item sits there reports `holding` (or `clear` if it moves), never a second `capture` — that's what stops a still-sitting item from being re-submitted every 250 ms.
- **Tokens invalidate stale results.** Every accepted capture increments `this.token`; `accepts(token)` checks both `valid` and an exact token match. Any movement (`invalidate()`) or a manual retry (`retry()`) bumps `armed`/clears `valid` and increments the token again, so a result that arrives late from the server for an item that has since moved or been swapped is rejected by `app.js`'s `current()` check (Section 6.4) rather than shown as if it belonged to whatever is now in frame.

`app.js`'s `tick()` is the glue: it calls `gate.update(sample, background, now)`, maps the returned `state` to a status message for the operator, and — only on `state === 'capture'` — calls `inspect(state.token)`; if the model is still busy with a previous request it instead calls `gate.retry()` so the *next* stillness period gets a fresh chance rather than being silently dropped.

8.3 Server side: rejecting bad frames before measuring

Even a "stable" frame per the browser's cheap RGB heuristic might not actually be a clean, correctly laid-out garment. `src/garmentiq/underwear_sizes.py` (../src/garmentiq/underwear_sizes.py) adds two model-informed gates, both raising `InspectionRejected` (caught in `phone_server.py` and turned into an "uncertain" result rather than an error — Section 6.3):

`check_inspection_mask(mask)` — runs on the segmentation mask, using OpenCV connected components:

```
count, _, stats, _ = cv2.connectedComponentsWithStats(binary, connectivity=8)
components = sorted(stats[1:], key=lambda row: row[cv2.CC_STAT_AREA], reverse=True)
if count < 2 or components[0][cv2.CC_STAT_AREA] < binary.size * 0.03:
    raise InspectionRejected("No clear garment; place one item in the inspection area.")
...
if area > binary.size * 0.90:
    raise InspectionRejected("Garment fills the view; leave visible background around it.")
if len(components) > 1 and components[1][cv2.CC_STAT_AREA] > max(binary.size * .015, area * .08):
    raise InspectionRejected("More than one object detected; remove hands, ruler and other items.")
...
if x <= margin or y <= margin or x + w >= width - margin or y + h >= height - margin:
    raise InspectionRejected("Garment touches the inspection boundary; show the entire item.")
```

In order: reject an empty/near-empty mask, reject a mask that's basically the whole frame (garment too close/zoomed, no background margin), reject a second sizeable connected component (a hand, the ruler, or a second item still in frame — small noise under the threshold is tolerated), and reject a garment touching the image boundary (partially out of frame).

`check_inspection_landmarks(landmarks)` — a basic geometry sanity check specific to the `shorts` landmark layout used for underwear:

```
points = {key: (...) for key in ("1", "2", "3", "4", "5", "7", "10")}
left, center, right = (points[key] for key in ("1", "2", "3"))
waist_width = right[0] - left[0]
if (waist_width <= 0 or abs(right[1] - left[1]) > waist_width * .20
        or not left[0] < center[0] < right[0]
        or points["5"][1] <= left[1] or points["7"][1] <= center[1]
        or points["4"][0] >= points["10"][0]):
    raise InspectionRejected("Lay the item flat with its waistband horizontal at the top.")
```

This checks that the waist-left/center/right landmarks (`1`, `2`, `3`) form a roughly horizontal line in the expected left-to-right order, and that the hem/crotch landmarks fall below the waist as expected for an upright, waistband-up layout — catching an upside-down or heavily rotated garment even if segmentation and detection both "succeeded."

Both checks are explicitly documented (in their own docstrings and in `LIVE_PRODUCTION_LINE.md`) as **heuristics, not proof of correctness** — they can't verify the object is actually underwear, or reliably catch folds/overlaps/blur/a convincing false segmentation. Don't strengthen the tolerance-matching logic (below) to compensate for a garment layout problem these checks should be catching instead — fix the check or the operating procedure.

8.4 The size decision — `estimate_size`

```
REFERENCES = {
    "M":   {"front rise": 11.24, "full length": 11.36, "hips": 14.62, "waist": 13.69},
    "L":   {"front rise": 11.14, "full length": 12.65, "hips": 16.72, "waist": 14.62},
    "XL":  {"front rise": 11.30, "full length": 12.16, "hips": 17.05, "waist": 15.79},
    "2XL": {"front rise": 12.28, "full length": 12.39, "hips": 18.09, "waist": 16.84},
}
DEFAULT_TOLERANCE_IN = 0.50
```

These four numbers per size are **measured, laid-flat garment dimensions in inches** from the team's own physical reference samples — not body measurements, not an industry sizing chart. Given a fresh `measurements` dict (four named distances, already converted to inches), `estimate_size`:

1. Validates every one of the four expected measurement names is present, in inches, and a finite positive number — any failure returns `uncertain(...)` immediately, never a guess.
2. For each of the four references, computes the per-dimension delta and an RMSE-in-inches distance, and flags it `within_tolerance` only if **all four** deltas are within `± tolerance_in` (default 0.50").
3. Sorts candidates by RMSE (closest first) purely for the response payload / UI's reference column — **the accept/reject decision is never "pick the nearest," only "is exactly one candidate within tolerance on every dimension."**
4. Exactly one match → `{"status": "matched", "size": "L", ...}`. Zero or more than one match → `{"status": "uncertain", "size": None, ...}` (with a reason distinguishing "no reference matches" from "measurements overlap multiple references").

This conservative design (see the module's own docstring: "*A nearest neighbour alone is not sufficient: unknown sizes must be rejected*") is deliberate and is directly tested — see `test_tolerance_boundary_and_no_forced_nearest_size` and `test_ambiguous_is_not_forced` in `test/test_underwear.py` (../test/test_underwear.py). **Do not "fix" a borderline-uncertain result by widening the tolerance** — that's explicitly called out as an anti-pattern in `LIVE_PRODUCTION_LINE.md`; instead treat it as a real manual-review case, or (if it's systematic) recheck the reference values themselves.

---

9. Deep dive: GPU / device management

`src/garmentiq/runtime.py` (../src/garmentiq/runtime.py) exists because the fork's primary target hardware is a **GTX 1050 Ti with only 4 GB VRAM**, too little to comfortably hold all three models (classification + segmentation + landmark detection) resident on GPU simultaneously at full resolution.

`select_device(requested)` does more than check `torch.cuda.is_available()` — it actually runs a tiny real convolution on the GPU before trusting it, because availability doesn't guarantee the installed PyTorch wheel actually supports this specific GPU's compute capability (this bit real users during CUDA-11.8-vs-Pascal-architecture wheel mismatches — see `docs/GTX1050TI_SETUP.md`'s "Why these versions?" section):

```
device = torch.device("cuda:0")
with torch.no_grad():
    x = torch.ones((1, 1, 8, 8), device=device)
    kernel = torch.ones((1, 1, 3, 3), device=device)
    torch.nn.functional.conv2d(x, kernel).sum().item()
return device
```

If that fails: an explicit `--device cuda` request raises loudly (fail fast, user asked for GPU and didn't get it); `"auto"` instead warns and silently falls back to CPU.

`InferenceRuntime` wraps a resolved device plus a `low_vram` policy, auto-enabled for GPUs with ≤ 4.5 GiB total memory:

```
if low_vram is None and self.device.type == "cuda":
    self.low_vram = torch.cuda.get_device_properties(self.device).total_memory <= 4.5 * 1024**3
```

- **`prepare(models)`** — called once at startup with all three loaded models. In low-VRAM mode it deliberately does *not* move anything to GPU yet (they stay on CPU/system RAM until each is actually used).
- **`run(function, model, **kwargs)`** — called around every actual forward pass (`classify`, `segment`, `detect`). Moves `model` onto the target device, runs `function`, and — in low-VRAM mode — moves it back off GPU (`model.to("cpu")`) and empties the CUDA cache in a `finally` block **regardless of success or failure**. This is what "only the model for the current stage is on GPU" (mentioned in the setup guide) actually means in code: classification, segmentation, and landmark detection each get swapped onto the 4 GB card just long enough to run, then swapped back off, trading extra PCIe transfer time for fitting at all.
- **Out-of-memory retry**: if a forward pass raises `torch.cuda.OutOfMemoryError`, `run()` falls back to CPU permanently for the rest of the process (`self.device = torch.device("cpu")`) and retries that one call on CPU — it does **not** retry CUDA later; only a process restart tries CUDA again. This means a single transient OOM (e.g. another app briefly grabbing VRAM) degrades the whole server to CPU speed until restarted — worth knowing when triaging "why did this get slow" reports.

`InferenceRuntime` also disables cuDNN's autotuner (`torch.backends.cudnn.benchmark = False`) — autotuning searches for the fastest kernel using extra workspace memory, which isn't a good tradeoff on a 4 GB card.

Segmentation size (`--segmentation-size`, one of 256/512/768/1024) is the other big VRAM lever, controlling BiRefNet's internal `resize_dim` (see `run_live.build_agent`). It defaults to 512 in low-VRAM mode, 1024 otherwise. Because `segmentation.extract()` always resizes the resulting mask back to the original image's pixel grid before returning it (Section 4.6), **changing this value never breaks ruler-pixel calibration** — it only trades mask precision (and therefore refinement/derivation accuracy) for speed and VRAM headroom. Always re-verify measurements against a physical tape measure after changing this, per the setup guide.

---

10. Setup, running, and operating the demos

Full step-by-step instructions already live in the two companion guides — this section is a condensed map so you know which document to open for what:

Need to...	See
Set up a fresh Windows PC with a GTX 1050 Ti from scratch	`docs/GTX1050TI_SETUP.md` (GTX1050TI_SETUP.md)
Understand GPU flags (`--device`, `--low-vram`, `--segmentation-size`) and troubleshoot CUDA/driver issues	`docs/GTX1050TI_SETUP.md` (GTX1050TI_SETUP.md#gpu-behavior-and-options) + `docs/GTX1050TI_SETUP.md` (GTX1050TI_SETUP.md#troubleshooting-and-updates)
Run the static photo-upload demo	`.\run_static_demo.bat` → `http://127.0.0.1:8080`
Run the live camera demo on the same PC	`.\run_phone_pc.bat` → `http://127.0.0.1:8080`
Run the live camera demo for a phone on the LAN	`.\run_phone_server.bat` → `https://<PC-LAN-IP>:8443`
Operate the paused-item underwear inspection station (calibration steps, station setup, gap timing)	`docs/LIVE_PRODUCTION_LINE.md` (LIVE_PRODUCTION_LINE.md)
Re-download or verify model weight checksums	`.\.venv\Scripts\python.exe scripts\download_models.py`
Run a full GPU + pipeline smoke test	`.\.venv\Scripts\python.exe scripts\check_runtime.py --require-cuda --smoke`

Two scripts worth understanding directly rather than just running:

- **`scripts/download_models.py`** — pins exact Hugging Face repo/revision/filename/SHA-256 tuples for the three required model files and refuses to proceed on a checksum mismatch, so a partial or corrupted download can never silently get used. `ensure_models(check_only=True)` (used by `check_runtime.py`) verifies without any network access — useful for offline environments once models are already present.
- **`scripts/check_runtime.py`** — reports Python/PyTorch/torchvision/CUDA versions, runs the same real CUDA convolution check as `select_device`, and (with `--smoke`) builds a real agent and fires real requests at both Flask apps' `/measure` endpoints in-process (via `app.test_client()`), asserting the response is HTTP 200 with finite inch measurements. This is the fastest way to confirm "the whole thing actually works end to end" on a new machine before handing it to an operator — run it after any dependency or model change.

---

11. Testing

File	Runner	Covers
`test/test_underwear.py` (../test/test_underwear.py)	`python -m unittest`	`estimate_size` reference matching (exact matches, tolerance boundaries, ambiguous/invalid rejection), `check_inspection_mask` / `check_inspection_landmarks` frame-quality gates, real measurement math through `LiveDetector` with a stubbed agent, and the full `phone_server` `/measure` API (auto-underwear mode, legacy mode, busy/429, failure/500 + lock release, static asset serving).
`test/test_runtime.py` (../test/test_runtime.py)	`python -m unittest`	`select_device` CPU/auto/explicit-CUDA behavior, `InferenceRuntime` low-VRAM offloading and OOM-retry-then-stay-on-CPU behavior, checkpoint-prefix stripping, classification following the model's own device, segmentation mask resize-back to original resolution.
`test/test_inspection.mjs` (../test/test_inspection.mjs)	`node --test`	`InspectionGate` state transitions: empty never triggers, a paused item triggers exactly once and its token is accepted, movement invalidates a pending/displayed result, the empty-gap rearm timing, a brief empty flash not being enough to rearm, manual retry, cumulative-drift detection, small-noise tolerance.
`test/test_live_page.mjs` (../test/test_live_page.mjs)	`node --test`	Page-level behavior with simulated camera/DOM adapters (not covered in this guide — read it directly alongside `app.js` if you're changing frontend control flow).
`test/*.ipynb`	Jupyter / Colab	Upstream GarmentIQ usage examples (classification, segmentation, landmark detection/refinement/derivation, tailor pipeline, custom instructions, training/fine-tuning) — reference material for the library API, not this fork's app layer.

Run everything from the repo root with the project's virtual environment:

```
.\.venv\Scripts\python.exe -m unittest discover -s test -p "test_*.py" -v
node --test test/test_inspection.mjs
node --test test/test_live_page.mjs
```

Node is only needed to run the two `.mjs` test files during development — it is **not** a runtime dependency of any shipped demo (the frontend JS runs in the browser, not under Node). None of these tests replace manually testing in a real browser with a real camera, nor a real accuracy trial with a tape measure — see Section 13.

12. Cookbook: how to make common changes

Each recipe names the exact file(s) to touch and what to watch out for.

Add or change a measurement for an existing garment type

Edit that garment's instruction JSON in `src/garmentiq/instruction/` (e.g. `shorts.json` (../src/garmentiq/instruction/shorts.json)). To add a measurement between two *already-predefined* landmarks, just add an entry under `"measurements"`:

```
"waist to hem": {"landmarks": {"start": "2", "end": "5"}, "description": "/"}
```

No Python changes needed — `compute_measurement_distances` picks up any measurement present in the JSON automatically. If you need a **new landmark** that isn't one of the model's predefined channels, see "Add a derived landmark" below; if you need a completely custom instruction (e.g. a subset of the default measurements only), follow the "Custom measurement instruction" (../README.md#custom-measurement-instruction) recipe in the main README — build a modified `garment_classes`-shaped dict pointing at your own JSON file, rather than editing the packaged one, if you want this to remain an opt-in override.

Add a derived (geometrically computed) landmark

Add a landmark entry with `"predefined": false` and a `"derivation"` block naming `derive_keypoint_coord` and five landmark IDs plus a `direction` (`"parallel"` or `"perpendicular"`) — see Section 4.7 for exactly what each ID means geometrically, and `derivation_dict.py` for the parameter schema `prepare_args` validates against. If you need a genuinely new *kind* of derivation (not just new parameters to the existing one), add a new function under `src/garmentiq/landmark/derivation/`, register its parameter schema in `derivation_dict.py`, and add a branch for it in both `prepare_args.py` and `process.py`'s dispatch table.

Change the underwear size references or tolerance

Edit `REFERENCES` and/or `DEFAULT_TOLERANCE_IN` in `src/garmentiq/underwear_sizes.py` (../src/garmentiq/underwear_sizes.py). **Also update the duplicated copy in `static_image_web/index.html` (../static_image_web/index.html)** (the `underwearReferences` JS object) — unlike `phone_web/app.js`, which fetches `/size-references` live, the static upload page hardcodes its own copy. Restart whichever server(s) are running afterward — these are read once at import time, not hot-reloaded. Re-run `test/test_underwear.py::SizeTests.test_known_reference_values_are_preserved` (or update it) after changing values, since it currently pins today's exact numbers.

Add a new underwear size (e.g. a size S or a second style)

`REFERENCES` is a plain dict keyed by size name — add a new key with its four measured dimensions. `estimate_size` and the frontend's reference table (`app.js`'s `loadReferences()`) both iterate the dict generically, so no other server-side code needs to change. Update `phone_web/index.html`'s hardcoded `['M', 'L', 'XL', '2XL']` list in `loadReferences()` if you want the new size to render in that specific column order (currently that list, not `Object.keys`, controls display order). Remember this feature only supports **one garment style at a time** — a second style with its own reference table would need its own `REFERENCES`-like

dict and a way for the UI/API to select between them (currently `mode=underwear` always implies `class_name="shorts"` and the single `REFERENCES` table).

Adjust automatic-inspection timing/sensitivity

All tunables are constructor defaults on `InspectionGate` in `phone_web/inspection.mjs` (../phone_web/inspection.mjs):

```
constructor({settleMs = 1250, emptyMs = 750} = {}) { ... }
```

plus the hardcoded thresholds inside `update()` (`foreground < .05`, `motion > .015`) and `changedFraction`'s `threshold = 22` (per-channel RGB difference). `app.js` constructs the gate with no options (`gate = new InspectionGate();`), so change the defaults here, or pass an options object at that call site for a one-off tweak. After any change, re-run `node --test test/test_inspection.mjs` — several tests assert on exact timing boundaries and will tell you immediately if a change altered behavior you didn't intend.

Change or add a frame-quality/geometry rejection rule

Edit `check_inspection_mask` or `check_inspection_landmarks` in `src/garmentiq/underwear_sizes.py` (../src/garmentiq/underwear_sizes.py). Both raise `InspectionRejected(message)` — that message is what the operator sees as the `status` text. `test/test_underwear.py::FrameTests` has synthetic-mask and synthetic-landmark test cases you can extend with your new scenario before wiring it into the real check.

Swap the segmentation backend (BiRefNet → SAM) in the live apps

Change `segmentation_model_class`/`segmentation_model_path`/`segmentation_model_args` in `run_live.build_agent()` (../run_live.py) to the SAM equivalents shown in the main README's "Segmentation" (../README.md#segmentation) quick-start (`SamModel`, `load_sam_config`, `load_sam_processor`, and an `input_points` prompt). You will need to decide how to supply `input_points` per-frame in a live setting (SAM needs a pixel coordinate believed to be inside the garment) — e.g. derive it from the calibrated ROI's center — since none of the current live code paths provide one.

Change the classification model

Change `classification_model_class`/`classification_model_path`/`classification_model_args` in `run_live.build_agent()` (../run_live.py) — e.g. to `CNN3` or `CNN4` from `garmentiq.classification.model_definition`, with weights from the model table in the main `README.md`. To fine-tune your own weights first, follow the "Classification model fine-tuning" (../README.md#classification-model-fine-tuning) recipe.

Change device/VRAM defaults

The three CLI flags (`--device`, `--low-vram`, `--segmentation-size`) are defined once in `run_live.add_runtime_arguments()` and consumed by `run_live.build_agent()` — change defaults there rather than in each entry point. See Section 9 for what each one actually does before changing a default that affects every deployed instance.

Add a new Flask API endpoint

Add a route to `phone_server.py` or `static_image_server.py` following the existing pattern: validate inputs early and return a 400 with a clear `error` message on anything malformed, acquire `inference_lock` (non-blocking, Section 5.3) around any call into the shared `agent`, and always release it in a `finally`. Add a corresponding `ApiTests`-style test in `test/test_underwear.py` (it already shows the `app.test_client()` + `unittest.mock.patch.object(..., "agent", fake_agent())` pattern for testing without loading real models).

Modify the calibration/inspection UI

Frontend logic lives entirely in `phone_web/app.js` (UI state, DOM, camera, HTTP) and `phone_web/inspection.mjs` (pure trigger logic, unit-tested). Keep that separation: anything that can be expressed as "given these RGB samples and a timestamp, what state are we in" belongs in `inspection.mjs` so it stays testable with plain `node --test`, without a browser or camera.

Publish a new upstream `garmentiq` PyPI release

Not typically relevant to this fork's deployment (the fork runs `src/garmentiq` directly via the editable install, not from PyPI), but `.github/workflows/publish.yml` builds and publishes to PyPI whenever a GitHub release tag starting with `py.` is published — bump `version` in `pyproject.toml` (../pyproject.toml) first.

---

13. Known limitations and non-goals

Worth internalizing before presenting this system as more capable than it is — most of this is already called out in the companion guides, condensed here for a full-codebase view:

- **This is an inspection prototype, not a validated production sorter.** It does not control a conveyor, trigger a PLC, reliably count items, or train a new neural network on your data.
- **The size decision is reference matching against four manually measured samples**, not a trained classifier and not a statistically validated tolerance. One sample per size is not enough to establish brand/style variation or accuracy — validate on multiple independently labelled garments per size before trusting it operationally.
- **Calibration is a single 1-D pixel/inch scale**, not a real camera calibration — it assumes a flat scene, negligible perspective/lens distortion, and a camera that never moves after calibration.
- **The browser's presence/motion detection is a cheap RGB-difference heuristic**, not a trained object tracker — patterned/moving belts, shadows, and lighting flicker can confuse it, and a near-identical item swapped in without a visible empty gap can be missed.
- **The mask/landmark rejection checks are heuristics**, not proof that the frame contains a correctly laid-out underwear item — they cannot reliably catch folds, overlapping garments, blur, hands, or a convincing false segmentation.
- **No inference-speed guarantee.** CPU inference can be much slower than a moving line; GTX 1050 Ti throughput needs measurement on the actual deployment PC. Nothing here promises video-frame-rate inference.
- **The bundled Flask servers are trusted-local-network development servers**, not hardened for the public internet — the phone server's HTTPS is an ad-hoc self-signed certificate for LAN camera permission only.

---

14. Appendix: key data structures

The "cleaned" garment dict (server responses, saved JSON, `clean_detection_dict` output)

```
{
  "shorts": {
    "class": "shorts",
    "landmarks": {
      "1": {"x": 611.0, "y": 861.0, "conf": 0.93},
      "20": {"x": 700.9, "y": 2070.0}
    },
    "measurements": {
      "waist":  {"distance": 615.0, "landmarks": {"start": "1", "end": "3"}},
      "hips":   {"distance": 615.4, "landmarks": {"start": "4", "end": "10"}}
    }
  }
}
```

- `landmarks`: keyed by 1-indexed landmark ID (string). `conf` is present only for detector-produced (predefined) points, not derived ones.
- `measurements`: keyed by human-readable name from the instruction JSON. `distance` is in **pixels** until divided by `pixels_per_unit`; after that division (done by `LiveDetector` / `annotate()`), a sibling `unit` key (`"in"` or `"px"`) is added.

`garment_classes` entry (`src/garmentiq/garment_classes.py`)

```
{
    "num_predefined_points": int,   # count of real (non-padding) channels for this garment
    "index_range": (start, end),    # 0-based, end-exclusive slice into the 294-channel HRNet output — f
    "instruction": "path/to/x.json" # the customizable landmark + measurement schema (Section 4.8)
}
```

`underwear_sizes.REFERENCES` size decision response (`estimate_size` return value)

```
{
    "status": "matched" | "uncertain",
    "size": "L" | None,
    "reason": "...",
    "tolerance_in": 0.50,
    "candidates": [
        {"size": "L", "rmse_in": 0.12, "within_tolerance": True, "deltas_in": {...}},
        ...  # all four sizes, sorted by rmse_in ascending
    ],
    "reference": {"front rise": 11.14, "full length": 12.65, "hips": 16.72, "waist": 14.62} | None
}
```

`InspectionGate.update()` return value

```
{state: "empty"|"moving"|"settling"|"capture"|"holding"|"clear", foreground: 0..1, motion: 0..1, token?:
```

`token` is present only on `"capture"`; pass it to `gate.accepts(token)` to check whether a result is still valid for whatever's currently in the inspection area.